

Assignments

Viewing assignment...

▼ Settings for "Project 1a F26"

Created by	Konstantin Laufer
Date created	Aug 27, 2026, 5:25 PM
Open	Sep 3, 2026, 6:15 PM
Due	Sep 14, 2026, 11:55 PM
Accept Until	Sep 16, 2026, 11:55 PM
Modified by instructor	Sep 4, 2026, 9:35 AM
Group Submission	No
For	Entire Site
Student Submissions	Inline only
Grade	Points (max 6.0)
Add due date to calendar	Yes
Alert:	No
Honor pledge:	Yes

Assignment Instructions

Individual project

Learning objectives

- Transition from Java to Scala
- An understanding of
 - object-oriented programming in Scala
 - algebraic data types and their representation as case classes
 - pattern matching
 - recursive functions
 - relationship with composite, decorator, and visitor patterns
 - test-driven development (TDD) in Scala and IntelliJ IDEA
 - Scala collections library and methods

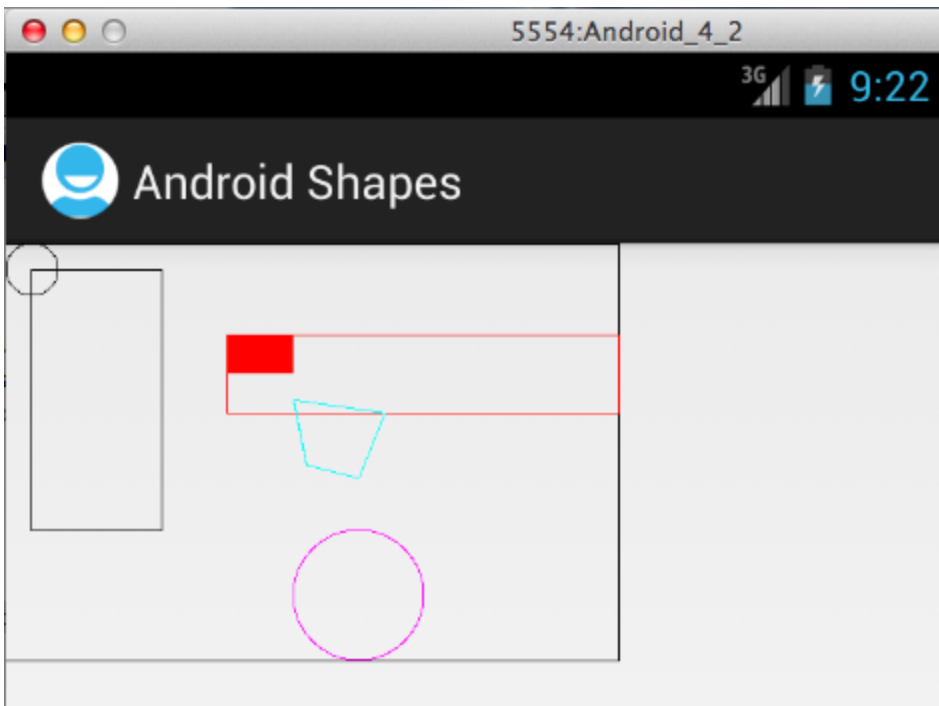
Functional requirements

- Copy the [shapes-oo-scala](#) example *using the "use this template" > "create new repository" dropdown button in the top right corner.*
- Study the code.
- Try to run the Main application and the unit tests. The code should compile, though most of the tests will fail.

```
sbt test:run  
sbt test
```

- Perform the steps marked TODO in the comments within the sources provided (View > Tool Windows > TODO in IntelliJ - or after installing a similar extension in VS Code). When you are done, all six bounding box tests should succeed.
- Following `boundingBox`, create a size behavior that counts the number of concrete leaf shapes (such as ellipses and rectangles) in a general shape.
- Following size, create a height behavior that computes the *height* of a shape tree (taking all kinds of shape nodes into consideration). [This used to be called *depth* in previous versions]
- In addition, create a scale behavior that (recursively) scales the dimensions of a shape by a given factor. (This affects shapes that have dimensions, including locations.)

See also this related Java/Android-based project from COMP 313/413: [description](#) [code skeleton](#) This corresponding image shows several types of shapes (including some features not required here, such as colors and polygons); the black rectangle surrounding all the other shapes is the *bounding box* for the entire group of those other shapes.



In addition, here is a related practice midterm exam, which should help conceptually and algorithmically if you did not work on the related Java/Android project: [practice exam](#) *The solution for this exam is attached below.*

Nonfunctional requirements

- Retain the original project structure.
- Keep your code as functional/immutable as possible.
- Use `map`, `foldLeft`, `sum`, and/or `min/max` in the completion of the `boundingBox` behavior.
- Using the existing test fixtures and following `TestBoundingBox`, create `TestSize`, `TestHeight`, and `TestScale` classes with at least the same number of tests but for your size, height, and scale behaviors, respectively.
- Use a suitable logging library to print debugging information (see also project 1b and the lecture notes).

Extra credit

- Graphical rendering of the shapes using [Doodle](#) or some other suitable graphics library. You should implement this capability as a new behavior `draw`.

- Automated unit testing of the draw behavior using offline drawing to instances of `BufferedImage` (see also [this example](#)).

Be sure to indicate in your project README if you attempted one or both of these extra credit tasks.

Submission

Your GitHub Classroom repository will be shared automatically with the instructor and TA. As you work on this project, keep your repo current by committing and pushing frequently. Once your project is ready to grade, submit a brief inline note to Sakai with the complete repo URL.

Grading criteria

Total 6 points:

- 0.5 add missing case classes
- 1 completion of `boundingBox` function
- 0.5 shape constructor validity checking *with suitable tests*
- 0.5 size function
- 0.5 height function
- 0.5 scale function
- 0.5 use of `map`, `foldLeft`, `sum`, `min/max`
- 0.5 `TestSize`
- 0.5 `TestHeight`
- 0.5 `TestScale`
- 0.5 configurable debug output using logging library (see lecture notes)

Possible deductions:

- 1 modification of existing code skeleton
- 1 missing AI transcript or affidavit that AI was not used

Additional resources for assignment

-  [PracticeExam.pdf](#) (156 KB; Aug 27, 2026, 5:25 pm)
-  [PracticeExamSolutions.txt](#) (6 KB; Aug 27, 2026, 5:25 pm)

▸ [Student view of the assignment "Project 1a F26"](#)

[Back to list](#)

Link



• Assignments

Short URL

<https://sakai.luc.edu/portal/directtool/d8401d87-82b3-4afe-905e-d6c2fbb2a279/>

